pyAmpliCol at one loop

Born–virtual interference: milestone status, benchmarks and architecture

Milestone 2026-10-24 · engine revision dfa6bf7c · AMD EPYC 9754 (shared host)

pyAmpliCol computes the interference $2\,\mathrm{Re}\left(B^{\dagger}GL\right)$ of one-loop and Born amplitudes with the UV and R_2 counterterms of the UFO loop model, for every squared coupling order and all three Laurent coefficients, in binary64, double-double (DD) and arbitrary precision (AP). The engine is the recursive *v2* engine: tree currents, loop topologies and loop numerators come from recursions, and no loop Feynman diagram is enumerated (Section 5). Counterterms come from the tree recursion (WP2.2); in production they will be plain tree vertices inserted exactly once, without loop matching (addendum J; `ct-simplify` in flight). This report records **gate G1**, the first end-to-end measurement of v2 against MadLoop, Recola and OpenLoops (Section 1), on the integration branch at dfa6bf7c (not yet on `loop`, which is at f9edf1db) and, as a development configuration, on the WP3.1 branch whose native per-point inputs are being integrated.

What the slice adds to `loop` (engine dfa6bf7c; merges since f9edf1db)

`a18b7599`	WP2.2	**Counterterms in the recursion.** R2 and UV counterterm trees from the tree recursion are the production route; site closure is the test oracle
`6735d3a5`	WP2.10	**Runtime pole selection.** poles = all | none | uv | ir | split with finite-only savings; NinjaRS 41a4c00 and oneloopmaster 5b710437
`dfa6bf7c`	Rust A0	**Contract-class reuse pass.** value equivalences only between currents of one evaluation contract; gg→ggggg CT-program reuse 190 → 43 s

On `loop` (f9edf1db): everything up to f9edf1db: v2 waves 0-1 with gate S1, gate G0, the native Rust executor (WP2.1), colour production (WP2.4), stability API (WP2.3), schedule M in Rust (WP2.8), tree-EW follow-up (WP2.9 part), the NJet reference, observables (WP2.5), the public v2 engine with plan v4 artifacts (WP2.6), requirements addendum F, G, H, J and the previous report. **Native build** `loop-native-poles-uvir-3dbd3a53` (digest `d9f854ae`…). **Measured here:** the slice with this package's timers (`work/v2-measure-g1`, build `loop-native-measure-g1-0f87abd2`) and the WP3.1 branch at `889af02c` (native inputs, build `loop-native-measure-g1-genscale-39824f9b`).

Performance gates: policy and status

- **Gate.** Faster than MadLoop and at most 1.30× each matched Recola and OpenLoops time, on the **finite part**, for processes from $2 \to 3$ upward. $2 \to 2$ is reported without a gate.
- **Protocol.** Providers in production mode with poles off where they allow it (OpenLoops `truepoles 0`, Recola without IR poles, MadLoop production build); pyAmpliCol finite-only (`poles = "none"`) with stability checks off, one pinned thread, the same frozen points.
- **Status (G1, Table 1).** The integrated slice fails every gated process (×11–28 of the limit): its per-point inputs still run in Python. With native inputs development 6 of 7 pass at the largest batch; $u\bar{u} \to d\bar{d}g$ fails (Table 3).
- **Seven legs** follow with the WP3.1 generator (`measure-g1-7leg`); no matched provider timing exists for $gg \to ggggg$: its speed is reported without a gate (Table 8, Table 13).
- **History.** The timings of 2026-10-18 measure the superseded per-diagram engine and appear only in Section 6.

G1: integrated slice fails 7 of 7 · native inputs[dev] pass 6 of 7

In flight (branches, not on the slice)

- **Generation at scale** (WP3.1 G-int, `work/v2-generation-scale`): native trees, Born and counterterm inputs, regeneration-free artifacts; timed here as the native-input configuration; audit and integration next.
- **Rust ports** (HANDOFF §0.4): native loop DAG (A1–A2), seeds (B), cells (C) and lowering (D), phase 0.
- **`ct-simplify`** (addendum J: no witness table or loop matching, `particle_veto`) and **`ct-assume-fix`** (the QED counterterm regression of a18b7599, found by the push gate).
- **`measure-g1-7leg`**: the seven-leg G1 rows after WP3.1; **electroweak** and complex mass (WP3.2) pending.

User decisions of 23 October

- **Counterterms** (addendum J): model R_2 and UV vertices are plain tree vertices inserted exactly once, admitted by coupling orders and a public `particle_veto`; no witness table or loop matching in production (site closure and witnesses: test oracles only).
- **Poles:** `"uv"` = UV poles of the bare loop, the counterterms' UV poles separate; `"ir"` = Catani poles of the renormalised virtual; one finite part.
- **Rust** (HANDOFF §0.4): heavy, contained generation stages move to rusticol (PyO3 API) with oracle equality; new heavy code in Rust.
- Generation may take O(hours) below 150 GB (addendum H); $gg \to ggggg$ speed without a gate, with rotation and Ward checks; no $2 \to 6$ before the $2 \to 5$ plan is complete.
- Binary64 observables: 10^{-12} unless the own estimate $E > 10^{-12}$ (then E, capped at 10^{-9}, plus DD within 10^{-24}).
- Stability-estimate tests: tenfold margin (error $\lesssim 10\,E$). Development first, proportionate testing; push to `loop` at milestones with this report.

1 Gate G1: end-to-end timings of the v2 engine

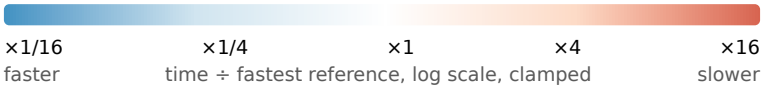
G1 times the complete Born–virtual evaluation, **finite part only**, through the call of the public v2 runtime (`NativeExecutor.evaluate`, which `V2VirtualRuntime` calls) with stability checks off, on one pinned core and one thread, binary64, 16 SIMD lanes, in batches of B distinct points of a frozen stream (REQUIREMENTS_ADDENDUM D, plan §5.8). Two pyAmpliCol configurations are timed:

- **integrated** — the integration branch as it stands (`work/v2-slice dfa6bf7c`, with this package's timers): trees, Born amplitudes and counterterm scalars are still formed per point in Python, then the loop stage runs natively;
- **native inputs** development — `work/v2-generation-scale 889af02c` (WP3.1, under integration), where the executor also forms the trees, the Born and the counterterm inputs itself (same timers; a scratch release build of that branch).

The providers run in production mode on the same stream: OpenLoops with `truepoles 0`, Recola without IR poles (one amplitude call per point), MadLoop at its production stability setting on exports built with MG5′s production flags; mean per point over the stream's first 16 points (MadLoop $gg \to gggg$: 8), on the core that timed the native-input configuration of the process. Providers have no batch API: one time per point serves every B. The stream's first three points are the reference campaign's, and pyAmpliCol's Born and finite part agree with every provider there and on all common points (Table 7).

Table 1: **Gate G1, full colour**, ms per point (median over the timed blocks), finite part, one thread. Rows $B = 1$ and the largest batch timed ($B = 128$; the integrated $2 \to 4$ rows only at $B = 1$: their Python inputs take seconds per point). Multiplier and colour: time ÷ the fastest provider of the row (legend below). Grey rows: generation — pyAmpliCol generation plus plan export with its peak memory; MadLoop graph generation, export and compilation; Recola process generation; OpenLoops: the prebuilt library. MadLoop $u\bar{u} \to gggg$ was not exported: the $gg \to gggg$ export took 2.4 h here (HANDOFF §0.1), and OpenLoops binds the gate. Gate: upper line integrated, lower line native inputs; the number is the time over the binding limit (MadLoop's time or 1.30× the faster of Recola and OpenLoops), so below 1 passes. Provider times repeated on a second core agree within 16 % pyAmpliCol's blocks spread by 17 % at most around their median (largest batch, native inputs).

Process	B	Integrated	Native inputs[dev]	MadLoop	Recola	OpenLoops	Gate
$gg \to gg$ $2 \to 2$, no gate	1	138 ×69	14.7 ×7.3	2.02 fastest	10.0 ×5.0	3.22 ×1.6	no gate
	128	127 ×63	2.22 ×1.1	2.02 fastest	10.0 ×5.0	3.22 ×1.6	no gate
	gen.	10.4 s · 0.3 GiB	9.82 s · 0.3 GiB	9.70 s	0.142 s	ppj	
$u\bar{u} \to d\bar{d}g$ $2 \to 3$	1	40.1 ×25	10.8 ×6.8	3.21 ×2.0	3.34 ×2.1	1.59 fastest	fail ×19 fail ×5.2
	128	31.5 ×20	2.63 ×1.7	3.21 ×2.0	3.34 ×2.1	1.59 fastest	fail ×15 fail ×1.3
	gen.	9.89 s · 0.3 GiB	10.6 s · 0.3 GiB	9.87 s	0.412 s	ppjjj	
$u\bar{u} \to u\bar{u}g$ $2 \to 3$	1	68.5 ×23	19.7 ×6.7	8.98 ×3.1	6.05 ×2.1	2.93 fastest	fail ×18 fail ×5.2
	128	53.9 ×18	3.80 ×1.3	8.98 ×3.1	6.05 ×2.1	2.93 fastest	fail ×14 pass 1.00
	gen.	10.4 s · 0.3 GiB	10.8 s · 0.3 GiB	10.9 s	0.468 s	ppjjj	
$u\bar{u} \to ggg$ $2 \to 3$	1	218 ×17	50.6 ×4.0	28.4 ×2.3	25.6 ×2.0	12.5 fastest	fail ×13 fail ×3.1
	128	178 ×14	11.2 ×0.89	28.4 ×2.3	25.6 ×2.0	12.5 fastest	fail ×11 pass 0.69
	gen.	12.3 s · 0.3 GiB	12.2 s · 0.3 GiB	17.6 s	0.634 s	ppjjj	
$gg \to ggg$ $2 \to 3$	1	2 790 ×29	211 ×2.2	247 ×2.6	351 ×3.6	96.9 fastest	fail ×22 fail ×1.7
	128	3 040 ×31	45.8 ×0.47	247 ×2.6	351 ×3.6	96.9 fastest	fail ×24 pass 0.36
	gen.	27.3 s · 0.5 GiB	14.8 s · 0.4 GiB	47.0 s	6.63 s	ppjjj	
$gg \to gggg$ $2 \to 4$	1	105 000 ×37	4 100 ×1.4	39 200 ×14	17 700 ×6.2	2 850 fastest	fail ×28 fail ×1.1
	128	not timed	985 ×0.35	39 200 ×14	17 700 ×6.2	2 850 fastest	— pass 0.27
	gen.	332 s · 4.1 GiB	96.1 s · 2.0 GiB	8 680 s	2 130 s	ppjjjj	
$gg \to t\bar{t}gg$ $2 \to 4$	1	7 190 ×15	840 ×1.8	5 450 ×11	1 990 ×4.2	476 fastest	fail ×12 fail ×1.4
	128	not timed	190 ×0.40	5 450 ×11	1 990 ×4.2	476 fastest	— pass 0.31
	gen.	61.4 s · 0.8 GiB	36.6 s · 0.7 GiB	218 s	61.4 s	ppttjj	
$u\bar{u} \to gggg$ $2 \to 4$	1	5 860 ×16	860 ×2.3	not exported	925 ×2.5	373 fastest	fail ×12 fail ×1.8
	128	not timed	190 ×0.51	not exported	925 ×2.5	373 fastest	— pass 0.39
	gen.	54.6 s · 0.7 GiB	39.4 s · 0.7 GiB	—	32.8 s	ppjjjj	



1.1 Where the time goes

Table 2: **Stage split**, ms per point, full colour, at the largest batch timed (integrated $2 \to 4$: $B = 1$). Trees/Born/CT: the per-point inputs — in Python for the integrated slice (trees, counterterm program, Born and counterterm colour projections, scalars), in the executor with native inputs. Numerator: seeds, fused factors and the dressing of the open-loop objects (the pure numerator evaluation). Scatter: closures into NinjaRS lanes. NinjaRS: the reduction without the masters; masters: the master integrals (canonical arguments, cache, oneloopmaster). Glue: point parameters, carrier conversion, assembly and the rest of the Python call. MadLoop (instrumented copy, 15 points; 3 for $gg \to gggg$): Born/counterterm HELAS calls, loop-numerator coefficients, reduction plus masters, residual (stability test, colour sums). Bar: shares, colours as the column heads.

Process		Trees, Born, CT	Numerator	Scatter	NinjaRS	Masters	Glue	Total	Shares
$gg \to gg$	native^{dev}	0.941	0.836	0.0886	0.114	0.0378	0.206	2.22	
	integrated	123	0.833	0.0890	0.112	0.0356	2.74	127	
	MadLoop	0.0315	1.27	—	0.585		0.107	1.99	
$u\bar{u} \to d\bar{d}g$	native^{dev}	0.394	0.754	0.468	0.137	0.111	0.771	2.63	
	integrated	27.0	0.751	0.463	0.136	0.111	2.62	31.5	
	MadLoop	0.0479	2.35	—	0.683		0.133	3.22	
$u\bar{u} \to u\bar{u}g$	native^{dev}	0.763	1.24	0.482	0.233	0.170	0.965	3.80	
	integrated	48.1	1.24	0.480	0.230	0.175	4.08	53.9	
	MadLoop	0.137	6.44	—	1.15		0.814	8.54	
$u\bar{u} \to ggg$	native^{dev}	5.85	2.94	0.480	0.468	0.228	0.535	11.2	
	integrated	168	2.91	0.470	0.452	0.213	5.69	178	
	MadLoop	0.346	18.5	—	2.54		2.87	24.3	
$gg \to ggg$	native^{dev}	28.3	11.8	0.538	2.05	0.487	2.76	45.8	
	integrated	3 010	12.0	0.533	2.08	0.490	10.5	3 040	
	MadLoop	2.42	184	—	11.3		39.5	237	
$gg \to gggg$	native^{dev}	696	221	2.42	47.9	5.85	5.35	985	
	integrated	101 000	3 830	4.34	38.8	5.80	145	105 000	
	MadLoop	88.9	18 600	—	209		17 700	36 500	
$gg \to t\bar{t}gg$	native^{dev}	102	45.9	0.923	20.4	6.31	15.0	190	
	integrated	6 370	757	3.33	20.0	6.56	27.2	7 190	
	MadLoop	24.2	3 290	—	105		1 740	5 160	
$u\bar{u} \to gggg$	native^{dev}	119	47.9	1.81	9.14	2.90	8.49	190	
	integrated	4 990	833	2.23	9.92	3.04	25.3	5 860	

Verdict of gate G1 (processes from $2 \to 3$; $2 \to 2$ without a gate).

- **Integrated slice: fails** every process, at ×11–28 of the binding limit at the largest batch and ×12–28 at $B = 1$. The per-point Python inputs take 85 %–99 % of its time (Table 2); the native loop stage alone is far below the limits.
- **Native inputs** development : at the largest batch it **passes** for $u\bar{u} \to u\bar{u}g$, $u\bar{u} \to ggg$, $gg \to ggg$, $gg \to gggg$, $gg \to t\bar{t}gg$ and $u\bar{u} \to gggg$ and **fails** for $u\bar{u} \to d\bar{d}g$ (×1.3 of the limit, OpenLoops binding); $u\bar{u} \to u\bar{u}g$ passes within the timing spread (1.00 of its limit; 0.87 at $B = 32$). At $B = 1$ with 16 lanes it fails everywhere (a one-point call pays a 16-lane chunk); with one lane per call it passes for $u\bar{u} \to ggg$, $gg \to ggg$, $gg \to gggg$, $gg \to t\bar{t}gg$ and $u\bar{u} \to gggg$.
- **Fastest of all tools** (aspiration): with native inputs at the largest batch pyAmpliCol is the fastest code for $u\bar{u} \to ggg$, $gg \to ggg$, $gg \to gggg$, $gg \to t\bar{t}gg$ and $u\bar{u} \to gggg$, and slower than OpenLoops for $u\bar{u} \to d\bar{d}g$ and $u\bar{u} \to u\bar{u}g$. NJet, not a matched provider, takes 4.58 s for its two-evaluation full-colour $gg \to gggg$ test (Table 11).
- Seven-leg rows ($u\bar{u} \to e^-e^+d\bar{d}g$, $u\bar{u} \to e^-e^+ggg$, $gg \to t\bar{t}e^-e^+g$, $u\bar{u} \to ggggg$, $gg \to ggggg$) need the WP3.1 generator and are added by the follow-on package `measure-g1-7leg`; $gg \to ggggg$ has no matched provider timing (addendum F, H): reported without a gate.

Table 3: **Levers**, measured on the rows of Table 2. Gains are measured where a column says so; otherwise they are the stage's share, an upper bound.

Lever	Evidence	Expected gain	Owner
Native per-point inputs (WP3.1 I3–I6: trees, Born, counterterm scalars in the executor)	Python inputs 85 %–99 % of the integrated time	measured: ×12–107 (integrated ÷ native, largest B)	`generation-scale` (G-int, under integration)
Compiled native trees (kernels as the tree engine's SymJIT planes, lanes batched)	native trees 15 %–71 % of the native-input time; $gg \to gggg$ 71 %	up to the share; ×3.4 for the largest	Rust ports A/B (`native-dag`, HANDOFF §0.4)
Lanes by batch in binary64 (`plan_export.lanes_for`: 16 lanes also for one point)	$B = 1$: one 16-lane chunk per call	measured: ×2.6–4.1 at $B = 1$ (16 lanes ÷ 1 lane)	executor policy (WP3.1 or wave 4)

Lever	Evidence	Expected gain	Owner
Finite-only result path (sum the virtual natively, no per-family lane export, Python assembly only for checks and observables)	glue 1 %–29 % of the native-input time	up to the share (largest at $2 \to 3$ with quarks)	`plan_export`, executor (wave 4)
Lane-major closures (no scatter, no second copy into the reducer's batch)	scatter 0 %–18 %	up to the share	executor, reducer adapter (wave 4)
AVX2 / AVX-512 kernels (runtime selection)	numerator 22 %–33 %	×1.0–1.4 on the dressing (gate G0)	wave 4 (WP4.6)
NinjaRS and masters: parallel reduction, transposed reduction, stage-2 sharing	NinjaRS + masters 5 %–14 %; masters 11 %–45 % of the reduction, oneloopmaster alone 2 %–10 %: the cache bookkeeping costs more than the integrals	one thread: up to the share; threads: DESIGN §5.6	NinjaRS requests (DESIGN §11.5)

The failing native-input row, $u\bar{u} \to d\bar{d}g$, takes 2.63 ms against a limit of 2.06 ms (OpenLoops 1.59 ms × 1.30). Its glue and scatter are 1.24 ms; without them it would take 1.39 ms, 0.68 of the limit. At $B = 32$ it takes 2.25 ms: its glue grows with the batch (0.320 ms per point at $B = 32$, 0.771 ms at $B = 128$).

1.2 Leading colour

Table 4: **Leading colour** (LC), ms per point, same protocol as Table 1; in each cell the full-colour (FC) time of the same configuration and LC/FC. No provider ratio: the providers time full colour. LC uses the committed QCD colour continuation (`examples/loop/source_colour`).

Process	B	Integrated		Native inputs[dev]	
$u\bar{u} \to d\bar{d}g$	1	40.4	FC 40.1 · 1.01	5.25	FC 10.8 · 0.49
	128	37.4	FC 31.5 · 1.19	1.91	FC 2.63 · 0.72
$u\bar{u} \to u\bar{u}g$	1	not timed		9.00	FC 19.7 · 0.46
	128	not timed		3.16	FC 3.80 · 0.83
$u\bar{u} \to ggg$	1	186	FC 218 · 0.85	22.8	FC 50.6 · 0.45
	128	177	FC 178 · 0.99	12.6	FC 11.2 · 1.13
$gg \to ggg$	1	2 320	FC 2 790 · 0.83	91.1	FC 211 · 0.43
	128	not timed		50.7	FC 45.8 · 1.11
$gg \to gggg$	1	not timed		2 070	FC 4 100 · 0.50
	128	not timed		1 090	FC 985 · 1.10
$gg \to t\bar{t}gg$	1	not timed		418	FC 840 · 0.50
	128	not timed		231	FC 190 · 1.21
$u\bar{u} \to gggg$	1	not timed		380	FC 860 · 0.44
	128	not timed		247	FC 190 · 1.30

LC is cheaper in the loop stage (fewer families and skeletons), but not yet end to end: with native inputs the trees at LC cost more than at full colour for the processes with many colour orderings ($u\bar{u} \to ggg$: 9.93 against 5.85 ms per point), and in the integrated slice the Python inputs dominate either way.

1.3 Throughput, model recalibration and schedule M

Table 5: **Kernel throughput and the design recipe** (native inputs, full colour, largest batch). Work: the executor's modelled complex multiply-adds of the chosen kernel forms per point (one lane); throughput: work × lanes over the dressing time (fused-factor builds excluded). Native loop: seeds, factors, dressing, scatter and reduction; outside: everything else of the call (trees, glue). Recipe: DESIGN §1.3, finite only, where the design has a row; measured ÷ recipe for the total and for the NinjaRS stage (masters included) against its model R.

Process	Work, MCMAC/pt	Dressing, ms	G CMAC/s	Native loop, ms	Outside, ms	Total, ms	Total ÷ recipe	NinjaRS ÷ R
$gg \to gg$	0.932	0.694	1.34	1.08	1.14	2.22	1.5–4.4	0.97–1.10
$u\bar{u} \to d\bar{d}g$	0.556	0.445	1.25	1.47	1.16	2.63	no row	—
$u\bar{u} \to u\bar{u}g$	1.09	0.879	1.24	2.13	1.67	3.80	no row	—
$u\bar{u} \to ggg$	3.18	2.44	1.30	4.12	7.07	11.2	no row	—
$gg \to ggg$	14.3	10.9	1.32	14.9	30.9	45.8	no row	—
$gg \to gggg$	268	215	1.25	278	708	985	2.4–7.1	1.05–1.24
$gg \to t\bar{t}gg$	56.0	43.2	1.29	73.5	117	190	1.2–3.5	0.99–1.39
$u\bar{u} \to gggg$	56.7	43.6	1.30	61.8	128	190	no row	—

Recalibration. The kernels run 1.24–1.34 G complex MAC/s per core on the real plans, the rate gate G0 measured on its synthetic programs (1.29–1.37); the dressing term of the recipe holds. The NinjaRS stage, masters included, lands on the design's model R (measured ÷ R 0.97–1.39, Table 5). What the recipe misses is the **outer factor**: it assumed trees, seeds and counterterms add 5–10 % (DESIGN §1.3), but with the current native inputs everything outside the native loop stage is 0.8–2.5 times the loop stage. `tools/loop_v2_costmodel.py --g1` prints the full comparison.

Table 6: **Schedule B against M end to end** (native inputs, one thread), ms per point, pure-gluon classes, the only classes where M merges seed entries (DESIGN §5.6). M/B below 0.85 would adopt M (the design's 15 % rule with the reduction included).

Process	Col.	B, $B=1$	M, $B=1$	M/B	B, largest	M, largest	M/B
$gg \to ggg$	FC	211	210	0.99	45.8	50.7	1.11
$gg \to ggg$	LC	91.1	84.8	0.93	50.7	46.6	0.92
$gg \to gggg$	FC	4 100	3 640	0.89	985	905	0.92
$gg \to gggg$	LC	2 070	1 780	0.86	1 090	1 020	0.94

B against M. End to end, M/B lies between 0.86 and 1.11 on the pure-gluon rows: M shortens the dressing of $gg \to gggg$, but the dressing is a minority of the time once trees and glue are counted (Table 2), so no row reaches the 0.85 that would adopt M; with a quark line M's entries do not merge at all (M/B 1.00 ± 0.02 on one thread for every quark-line class, DESIGN §5.6). **Decision (G1):** schedule B for every class at $n \leq 6$; M stays the planned default from $n \geq 8$ and is re-measured at seven legs with the WP3.1 plans (`measure-g1-7leg`).

Table 7: **Values at the timed points**: pyAmpliCol's finite part against each provider's production-mode value on the common stream points (the providers read the binary64 decimal text of the exact points), metric $|a-b| \,/\, \max(|a|,|b|,|B|\ \alpha_s/2\pi)$; Recola's finite part converted with the universal double pole (it returns no poles in production). Worst point: the stream point with the largest deviation, evaluated again with one rotation and the reduction-direction check; E is pyAmpliCol's returned relative accuracy estimate there (DESIGN §10.4), to be set against the deviation. Campaign: the first three points against the committed references (`tests/fixtures/loop/required`, native inputs), fixture metric and tolerance.

Process	Points	vs MadLoop	vs Recola	vs OpenLoops	Worst point: E	Campaign
$gg \to gg$	16	1.7×10^{-12}	7.4×10^{-13}	7.5×10^{-13}	—	no reference
$u\bar{u} \to d\bar{d}g$	16	6.8×10^{-13}	3.3×10^{-10}	6.2×10^{-13}	—	3.6×10^{-10}
$u\bar{u} \to u\bar{u}g$	16	3.4×10^{-12}	7.2×10^{-11}	6.8×10^{-12}	—	8.6×10^{-11}
$u\bar{u} \to ggg$	16	1.4×10^{-12}	1.5×10^{-12}	8.4×10^{-11}	—	no reference
$gg \to ggg$	16	1.7×10^{-6}	1.7×10^{-6}	1.7×10^{-6}	point 10: 9.9×10^{-6}	no reference
$gg \to gggg$	8	1.7×10^{-7}	4.0×10^{-7}	4.0×10^{-7}	point 1: 4.8×10^{-6}	4.0×10^{-7} above 2.0×10^{-8}
$gg \to t\bar{t}gg$	16	7.6×10^{-9}	1.5×10^{-9}	1.5×10^{-9}	point 13: 5.6×10^{-8}	1.1×10^{-11}
$u\bar{u} \to gggg$	16	—	9.4×10^{-12}	1.9×10^{-10}	point 9: 2.7×10^{-9}	no reference

Every deviation above 10^{-9} lies at a point whose returned estimate E exceeds it: binary64 losses at ill-conditioned points, which the stability checks flag and DoubleFloat resolves (Table 9), not disagreements between the codes. The deviations near 10^{-10} against Recola alone are Recola's own integral accuracy (COLLIER's default target 10^{-8}, HANDOFF §7); MadLoop and OpenLoops agree with pyAmpliCol to better than 10^{-11} on those points.

2 Required processes: references and generation

The plan requires the processes of Table 8 (plan §5.7), compared with MadLoop and Recola, and with OpenLoops where a public library exists. A reference provider is run only if its generation completes in about 45 minutes, or in a time comparable to pyAmpliCol's own generation of the process (HANDOFF §0.1, addendum F). Otherwise pyAmpliCol is validated by Ward and gauge identities, Catani poles, LC/NLC/FC and DD/AP consistency, stability checks, lower multiplicities and existing references.

Table 8: **Generation on this host.** Providers: MadLoop graph generation, export and compilation; Recola process generation; OpenLoops library availability (HANDOFF §0.1). pyAmpliCol: v2 generation + plan export (`LoopSlice.generate`, then plan v4 JSON), plan size and peak resident memory, one thread; a **development** measurement on `work/v2-generation-scale`, not part of the acceptance campaign. Values stand side by side without a ratio.

Process	pyAmpliCol v2, dev.	MadLoop	Recola	OpenLoops	45-minute rule
$gg \to ggggg$	358 s + 109 s 185 MB · 5.8 GiB	> 17 h graph generation alone; colour factors ≈ 89 days (MG5 estimate)	> 57 h, unfinished	no public library	stopped 22 Oct; not to be restarted
$u\bar{u} \to ggggg$	193 s + 69 s 70 MB · 2.4 GiB	12.2 h (5 793 + 38 040 + 122 s)	2.8 h (10 220 s)	no public library	over 45 min; results exist and are used
$gg \to gggg$	26.7 s + 5.8 s 15 MB · 0.55 GiB	2.4 h (285 + 8 354 + 21 s)	34 min (2 036 s)	prebuilt ppjjjj	MadLoop over 45 min; results exist and are used
$gg \to t\bar{t}e^-e^+g$ EW	not generated	11 min	22 s	pplttj_ew	within 45 min
$gg \to t\bar{t}e^-e^+g$ QCD	10.1 s + 3.6 s 2 MB · 0.2 GiB	2.7 min	29 s	pplttj	within 45 min
$gg \to t\bar{t}gg$	20.9 s + 2.9 s 5 MB · 0.29 GiB	3.6 min	63 s	ppttjj	within 45 min
$u\bar{u} \to e^-e^+ggg$	9.9 s + 3.7 s 2.3 MB · 0.22 GiB	2.4 min	16 s	pplljjj	within 45 min

Process	pyAmpliCol v2, dev.	MadLoop	Recola	OpenLoops	45-minute rule
$u\bar{u} \to e^-e^+d\bar{d}g$	9.5 s + 3.4 s 1.1 MB · 0.2 GiB	56 s	8 s	pplljjj	within 45 min
$2 \to 2, 2 \to 3$ controls	—	< 1 min	< 2 s	—	within 45 min

Table 9: **Validation of the required set** (development , `work/v2-generation-scale`; binary64, all three Laurent coefficients, three exact points per process, trees, Born and counterterms from the Python per-point inputs). Deviation metric $|a-b| \,/\, \max(|a|,|b|,|B|\ \alpha_s/2\pi)$. Points that the stability estimate flags in binary64 are resolved in DoubleFloat. ML: MadLoop; OL: OpenLoops.

Process	References	Agreement
$gg \to gggg$	Recola, OL	Recola, OpenLoops, all coefficients: ≤ 8.9×10⁻¹² at point 0 and ≤ 4.4×10⁻¹¹ at point 2 (largest: finite part); point 1: 8.4×10⁻⁹ in binary64 (flagged, estimate 3.0×10⁻⁶), 7.1×10⁻¹¹ in DoubleFloat
$gg \to t\bar{t}gg$	ML, Recola, OL	MadLoop, OpenLoops ≤ 2.1×10⁻¹¹; Recola ≤ 1.0×10⁻¹⁰
$u\bar{u} \to e^-e^+ggg$	ML, Recola, OL	all three providers ≤ 6.9×10⁻¹³
$u\bar{u} \to e^-e^+d\bar{d}g$	ML, Recola, OL	all three providers ≤ 8.5×10⁻¹³
$gg \to t\bar{t}e^-e^+g$ QCD	ML, Recola, OL	≤ 3.6×10⁻¹² at two points; third point 5.7×10⁻⁹ in binary64 (estimate 1.7×10⁻⁹), MadLoop to 1.9×10⁻¹⁴ in DoubleFloat
$gg \to t\bar{t}e^-e^+g$ EW	ML, Recola, OL	not started: Feynman-gauge massive vectors, Goldstones, complex-mass scheme, generic EW counterterms (WP3.2)
$u\bar{u} \to ggggg$	ML, Recola	MadLoop ≤ 2.6×10⁻¹², Recola ≤ 6.1×10⁻¹⁰ (finite ≤ 2.9×10⁻¹⁰), all coefficients, three points
$gg \to ggggg$	NJet (no gate)	Born and poles against NJet (§3); finite part waits for the native counterterms

Table 10: **Provider references for the seven-leg gates**, s per point, one pinned core, production mode (finite part; OpenLoops `truepoles 0` with I-operator poles, Recola without IR poles, MadLoop production build), measured on 2026-10-20 for the design (DESIGN §1.3, Table 1b); inferred values of that table are omitted. G1 re-measures them with the seven-leg rows (`measure-g1-7leg`); the $2 \to 3$ and $2 \to 4$ references are in Table 1. Threshold: the largest pyAmpliCol finite-only time that would pass, 1.3× the fastest matched Recola/OpenLoops time where it is below MadLoop's.

Process	OpenLoops	Recola	MadLoop	Threshold
$u\bar{u} \to e^-e^+ggg$	0.348–0.353	0.246–0.259	1.41	0.32
$u\bar{u} \to e^-e^+d\bar{d}g$	0.053–0.055	0.035	0.143	0.045
$gg \to t\bar{t}e^-e^+g$ QCD	0.425–0.433	0.59–0.67	3.03–3.04	0.55
$gg \to t\bar{t}e^-e^+g$ EW	0.96–1.12	0.95–1.02	6.90–7.09	1.24
$u\bar{u} \to ggggg$	no library	not measured	not measured	—
$gg \to ggggg$	no library	none (§0.1)	none (§0.1)	no gate

3 Seven gluons: NJet and pyAmpliCol

3.1 NJet: Table 5 of arXiv:1209.0100, re-measured on this host

NJet is the independent reference for $gg \to ggggg$, where MadLoop and Recola cannot generate under the 45-minute rule and OpenLoops has no library. It is massless five-flavour QCD, and it is not one of the gate's matched providers.

Table 11: **NJet, s per point**, double precision, two evaluations each (NJet's scaling test). Paper: Xeon E3-1240 at 3.3 GHz, NJet 1.0. This host: one pinned core, idle at launch, of the shared host; 10 frozen points per channel, mean (spread ≤ 2 %); the multiplier is time ÷ the paper's. De-symmetrised sums equal the standard sums only after phase-space integration; they are timing figures, not point values.

Channel	Table 5	NJet 3.1.2		NJet 2.1.1		Table 5	NJet 3.1.2		NJet 2.1.1	
	standard full-colour sum					de-symmetrised sum				
$gg \to gg$	0.03	0.00295	×0.1	0.00291	×0.1	0.03	no class		no class	
$gg \to ggg$	0.22	defective channel		0.141	×0.6	0.07	defective channel		0.0452	×0.6
$gg \to gggg$	6.19	4.59	×0.7	4.58	×0.7	0.57	0.39	×0.7	0.38	×0.7
$gg \to ggggg$	171.31	142	×0.8	246.7	×1.4	3.07	2.41	×0.8	4.19	×1.4

At seven gluons one NJet 3.1.2 evaluation without the scaling test takes 71.2 s (standard) and 1.22 s (de-symmetrised), at 69 MB; double-double costs 1 990 s per point. NJet 3.1.2's numerical five-gluon channel is 9/2 too large (NJet 2.1.1 is used there), and at the unrotated seven-gluon points NJet keeps only 4–6 digits in double precision (NaN at 3 of 10), so the reference values are taken at exactly rotated points.

3.2 pyAmpliCol development measurements

These are development measurements on the branches named, on the shared host. Generation and the counterterm program ran in one thread; the bare-loop job's thread count was not recorded, so its wall times are not per-core timings (Table 12). They are not

part of the acceptance campaign and not a like-for-like speed comparison: the native path below does not yet include counterterms or R_2.

Table 12: ***gg → ggggg* in full colour, development runs.** Wall times and peak resident memory from `/usr/bin/time` for the whole process, including model loading. Generation and the counterterm program are single-threaded (CPU share 101 %, 101 % and 99 %; `RAYON_NUM_THREADS=1` for the counterterm program). The bare-loop job did not record its thread count (the native executor's default is all cores); the whole job, generation and export included, averaged 168 % CPU. Its 1 769 CPU-s (user + system), less one core over generation (471 s) and export with trees (418 s), leave at most 879 CPU-s for the 146 s of evaluations: up to 6.0 busy cores on average and up to 176 CPU-s per point, so the per-core comparison with NJet is undetermined. Counts of the generated slice: 9 414 families, 810 495 skeletons, 197 539 trie nodes, 4 718 tree blocks, 1 080 colour-seed classes on a 720-element Born basis, 4 837 counterterm sites.

Run (branch)	Stages	Wall	Peak memory
Generation and plan, first run `work/v2-generation-scale`, 03:55 UTC	`LoopSlice.generate` 7.4 min (peak 4.2 GiB), plan export 5.4 min, JSON 38.7 s; 949 MB plan	13.6 min	9.3 GiB
Generation and plan, latest run same branch, 10:42 UTC	`LoopSlice.generate` 6.0 min (peak 4.2 GiB), plan export 1.8 min, JSON 9.05 s; 185 MB plan (derived row links)	8.0 min	5.8 GiB
Counterterms in the recursion `work/v2-ct-recursion`	Plain blocks 9.4 min, CT program 53.5 min; 3 070 042 CT currents, 34 781 532 interactions, 847 root sites, no rule left to site closure; script total 63.4 min. Root-closure (bind) terms not measured; run before the request-scoped lowering (ct-recursion DESIGN §6.1)	66.4 min	52.1 GiB
Bare loop + Born, native trees `work/v2-generation-scale`, 5 points	Binary64; per point the point and one rotation (two evaluations); no counterterms, no R_2; thread count not recorded (up to 6.0 cores, ≤ 176 CPU-s per point)	25.5–41.3 s/pt	18.5 GiB whole job

On the same host (Table 11) NJet takes 142 s per point for its standard sum and 2.41 s for its de-symmetrised sum, two evaluations each, on one pinned core, idle at launch, of the shared host. The native path's 25.5–41.3 s per point (also two evaluations) are wall times of a multi-threaded job: its CPU accounting allows up to ≈6.0 busy cores, ≈176 CPU-s per point, during the evaluations, which is above NJet's 142 s. Whether the native bare path is faster than NJet's standard sum per core is therefore undetermined until a one-thread run exists. The comparison is not like-for-like either: counterterms and R_2 are missing on the pyAmpliCol side, its job ran unpinned and multi-threaded on the same shared host, and the de-symmetrised sum is not a point value. Generation fits addendum H's budget (O(hours), below 150 GB). MASTER_PLAN quotes the same two peaks as KiB/10^6 ("9.7 GB", "54.6 GB"); of the run records, the counterterm record prints KiB/10^6 and the generation record GiB.

3.3 Validation without a matched provider

Table 13: **Checks of the seven-leg QCD results**, development measurements on `work/v2-generation-scale`, not part of the acceptance campaign, except the Ward row (WP2.5 on `loop`); binary64, deviation metric of Table 9. NJet values: fixture `gg_ggggg_njet.json` (NJet 3.1.2 double-double at exactly rotated points, scaling-test estimates ≤ 1.3×10^{-24}). The bare single pole is compared after subtracting the counterterm pole of its request, as marked in each row: $(\alpha_s/4\pi)B[(n-2)\beta_0^{(5)}+8T_F/3] = (\alpha_s/4\pi)B\cdot 119/3$ for loop_sm (five-flavour coupling renormalisation and the top's decoupling term) and $(\alpha_s/4\pi)(n-2)\beta_0^{(5)}B = (\alpha_s/4\pi)B\cdot 115/3$ for the top-free request.

Check	Result	Status
$gg \to ggggg$, native bare loop, 5 points: double pole ÷ Born against $-(\alpha_s/2\pi)\cdot 7C_A$	≤ 5.4×10^{-12}; rotation estimates 1.6×10^{-10}–2.9×10^{-6}	passed
$gg \to ggggg$ against NJet, native bare loop (loop_sm, 5 points; shift 119/3)	Born ≤ 1×10^{-13}, double pole ≤ 5.4×10^{-12}, single pole 3.5×10^{-12}–8.3×10^{-10} at points 0–3; 7.2×10^{-8} at point 4, which pyAmpliCol's estimate (2.9×10^{-6}) flags	no deviation beyond the estimates
$gg \to ggggg$ top-free (NJet's scheme) against NJet, native bare loop, 3 points; shift 115/3	Born ≤ 4.9×10^{-14}, double pole ≤ 5.4×10^{-12}, single pole 3.4×10^{-9} and 5.2×10^{-9} (estimates 1.4×10^{-8}, 5.0×10^{-8}), 1.4×10^{-11}	finite part needs the native counterterms
Top-free full virtual against NJet at lower multiplicity	$gg \to gg$: ≤ 2.1×10^{-14} at 10 points, full virtual; $gg \to ggg$: ≤ 2.3×10^{-13} at the stable points; the flagged point 3×10^{-14} in DoubleFloat; $gg \to gggg$: 1.0×10^{-11} at point 0; point 1 1.6×10^{-7} in binary64 (estimate 2.7×10^{-6}), 4.7×10^{-15} in DoubleFloat (`test_loop_v2_topfree.py`)	passed
$u\bar{u} \to ggggg$ full virtual against MadLoop and Recola, 3 points	MadLoop ≤ 2.6×10^{-12}, Recola ≤ 6.1×10^{-10} (finite ≤ 2.9×10^{-10}); the providers agree to 6.2×10^{-10}	passed
Ward identity, one gluon's $\varepsilon \to p$, at seven gluons	asymmetric Ward mode of WP2.5 (loop side only, bare + UV + R_2); passes on $gg \to gg$ and $u\bar{u} \to ggg$ (`test_loop_v2_observables.py`)	not yet run at seven gluons (addendum H)

First $gg \to ggggg$ virtual development `work/v2-generation-scale`, not part of the acceptance campaign (full colour, Born α_s^5, loop_sm with top loops, point 0, $\mu = 1$ TeV; Python per-point inputs with counterterms, about 3 h per evaluation on that development path):

	Born	finite	$1/\varepsilon$	$1/\varepsilon^2$
value, GeV^{-6}	3.62326×10^{-6}	-1.30223×10^{-5}	-6.93874×10^{-6}	-1.42896×10^{-6}
rotated copy, rel. diff.		5.9×10^{-9}	2.6×10^{-9}	2.5×10^{-13}
NJet (no top), deviation	3.8×10^{-14}	top loops	1.2×10^{-9}	1.1×10^{-12}

The finite part differs from NJet's by the top-loop contribution (-1.34×10^{-8}, i.e. $-0.197\ |B|\ \alpha_s/2\pi$), which continues the sign and size seen at four to six legs; it is not a test.

4 Capabilities at this milestone

Table 14: **Capability matrix** of the v2 engine on the slice at dfa6bf7c, with branch work marked.

Feature	Status	Boundary
Recursive engine	supported	Trees, families and loop numerators from recursions; no loop diagram is enumerated (FeynKit graphs are test oracles only). Counterterms: see below.
Full, NLC and LC colour	supported	Exact seed tables in N_c; LC/NLC by generated classes and truncated cells. LC is cheaper in the loop stage but not yet end to end at large batches (its native trees cost more, Table 4), nor to generate at seven legs.
Colour and spin correlations	supported	Directed colour histories at LC/NLC/FC, spin sources, resolved helicities, flavour aliases, Ward sources (WP2.5). Seven-leg directed LC/NLC tables need the Rust seed tables (WP3.1).
Per-order partials	supported	Every squared coupling order at tree level and at one loop, mixed Born orders, per-correction loop orders.
Poles at runtime	supported	`poles = "all" | "none" | "uv" | "ir" | "split"` (WP2.10, on the slice since 6735d3a5): finite-only mode with the finite part bitwise unchanged and the pole work skipped; UV poles of the bare loop and IR (Catani) poles of the renormalised virtual with the user's semantics of 23 October.
Stability estimates	supported	Rotation checks (any number, 0 = estimate absent), reduction direction, exact rescaling, in any combination; no automatic escalation (WP2.3).
Precision carriers	supported	Binary64, DoubleFloat, arbitrary precision and exact rationals in the native executor; DD/AP point inputs are formed in Python Decimal until WP3.1.
Native executor, schedules	supported	Rust, 16 lanes (also for a one-point call, a G1 lever), threads; schedule B for every class at $n \le 6$ (G1 decision, Table 6), M native, F as oracle. The NinjaRS reduction runs serially. Development report: stage times, modelled kernel work, master-integral time.
Plan v4 artifacts and API	partial	Published and loaded through Python, TOML and CLI; relocation to another path tested on the same host and CPU. Loading regenerates the process (the `LoopSlice`) in Python and checks the plan digest, which addendum G forbids for prepared artifacts; WP3.1 removes it. The Rust/C/C++/Fortran SDK refuses v4 until WP3.1.
Counterterms (model UV, R_2)	partial	Counterterms from the tree recursion are the production route on the slice (WP2.2, a18b7599), site closure the test oracle; both still admit sites by the witness table. Production rule (addendum J): plain tree vertices inserted exactly once, admitted by coupling orders and `particle_veto`, no loop matching; `ct-simplify` removes the matching. Per point they are formed in Python on the slice, natively on the WP3.1 branch.
Direct R_2	deferred	A bonus by the user's rule; bounded controls only.
Electroweak, complex mass	open	Tree EW lanes fixed, the Dirac–Weyl class refused; loop EW, Goldstones and CMS are WP3.2.
Required set	in progress	On the WP3.1 branch seven of eight generate and six agree with their provider references in all coefficients (Table 9); $gg \to ggggg$ is validated in Born and poles; the EW target is not started.
Performance gates	in progress	G1 measured (Section 1): the integrated slice fails every $2 \to 3$ and $2 \to 4$ gate (Python per-point inputs); with the WP3.1 native inputs six of seven pass at the largest batch, $u\bar{u} \to d\bar{d}g$ fails by 1.3×, $B = 1$ needs lane adaptation. Seven legs: `measure-g1-7leg`.
$gg \to 6g$	deferred	No $2 \to 6$ work until the $2 \to 5$ plan is complete (user, 23 October).

5 Architecture of the v2 engine

5.1 The computed quantity

For Born order a, loop order b and squared coupling order $s = a + b$,

$$V_{s,k} = \mathcal{N} \sum_{a+b=s} 2\,\mathrm{Re}\left[B_a^\dagger G_{BL} L_{b,k}\right], \quad k \in \{0,-1,-2\}, \quad V = C_\varepsilon \left(V_0 + \frac{V_{-1}}{\varepsilon} + \frac{V_{-2}}{\varepsilon^2}\right), \quad C_\varepsilon = \frac{(4\pi)^\varepsilon}{\Gamma(1-\varepsilon)},$$

where G_{BL} is the colour metric between Born and loop colour structures and $\mathcal{N}$ holds the initial-state averages and identical-particle factors (MadLoop/BLHA convention; couplings included, nothing divided by the Born). Directed colour and spin correlations replace $2\,\mathrm{Re}(\ldots)$ by the two cross terms $B_A^\dagger G L_B + L_A^\dagger G B_B$.

5.2 Pipeline

For every denominator family f, sector s and observable column o, v2 builds the four-dimensional numerator $N_{f,s,o}(q) = \sum_\alpha c_\alpha q^\alpha$ that NinjaRS reduces: Born-weighted and helicity-summed, produced by three recursions that share all work (Figure 1).

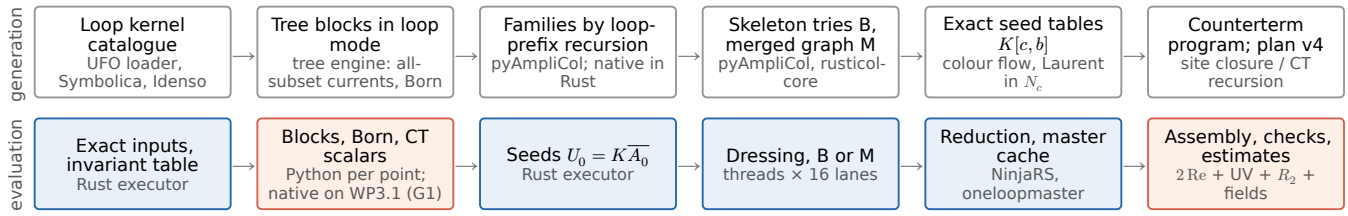


Figure 1: **v2 pipeline.** Upper lane: generation, once per process, into a plan v4 artifact. Lower lane: evaluation per batch of points. Blue: native batched numerics in the Rust executor; orange: still formed in Python per point (native lanes on the WP3.1 branch); white: generation. The owning code or dependency is named under each step.

Trees. The tree engine's colour-ordered Berends–Giele recursion computes, once per point, every off-shell current over every leg subset a loop vertex can absorb (the *blocks*), the Born colour components A_0 and, with WP2.2, every tree with one counterterm vertex. A loop-aware reachability test keeps exactly the currents some family position or root needs. Blocks with transverse sources use the exact transverse Yang–Mills kernels; loop segments, counterterm currents and Ward sources use the full ones.

Families and skeletons. A Recola-type recursion over loop prefixes (ordered leg sets, propagator species and masses) yields the denominator families, keyed by the canonical cyclic sequence of (leg mask, mass) and equal to Recola's tensor-integral identities. Per family it yields the colour-ordered *skeletons*: rings of blocks, each block summing all trees on its leg subset (Figure 2). Skeletons are never stored as a list; they are the leaves of the schedule tries. Only certified structural zeros are dropped (scaleless rings, external wave-function corrections, VEV tadpoles, colour zeros); nothing is restricted to 1PI.

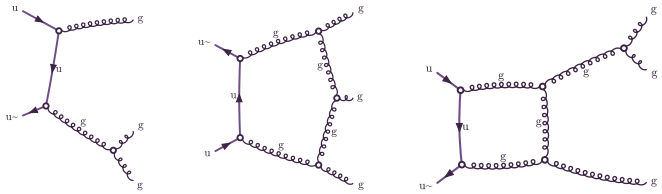


Figure 2: Three graphs of $u\bar{u} \to ggg$ drawn by FeynKit's renderer from its own generation (an oracle, not a production input). In v2 the Born (left) is a closure of blocks; the pentagon (middle) is a ring of five single-leg blocks; the box with a tree attachment (right) is a ring of four blocks, one of them the two-leg current, computed once per point and shared by every skeleton that contains that leg subset.

Loop numerators and schedules. An open-loop recursion dresses skeletons one segment at a time with polynomial-valued objects, $O' = O \cdot Y + \mathrm{raise}\big(O \cdot Z_\mu, q^\mu\big)$, in the NinjaRS monomial layout ($\binom{r+4}{4}$ coefficients at rank r, no rank cap). The seed is the colour-summed Born interference, and each block's helicities are summed as it is attached. One generator feeds three schedules (Table 15).

Table 15: **Three schedules on one generator** (DESIGN §2.2, §5.6).

	F: forward	B: suffix trie (default)	M: merged, primitive-seeded
Born weighting	at closure	at the leaves (one seed per skeleton class)	at the cut, one seed per colour ordering
Shared	prefixes across families	all prefixes with equal family and remainder	prefixes across families and compositions
Colour certificate	none needed	none needed	certified classes: adjoint rings, traces, quark rings
Role	oracle	default; generic fallback	pure-gluon classes; $n \geq 8$ default

Colour. Colour lives only in exact seed tables $K[c,b]$, Laurent polynomials in N_c between seed classes c (the colour vector obtained by closing the cut of a skeleton's colour flow) and Born basis elements b; per point the seeds are $U_0 = \sum_b K[c,b]\overline{A_0[b]}$, with a symmetric-group FFT where certified. LC and NLC differ from full colour only in which classes are generated and how cells are truncated: at LC each colour ordering is seeded by its own Born amplitude and skeletons below the leading N_c degree are never generated. Certificates (adjoint rings, fundamental traces, quark rings) are proved at loop level, never inherited from the tree level.

Reduction and masters. Families close directly into NinjaRS `TensorNumerator` lanes: one reduction per (family, sector, column), families with equal (propagators, rank) batched as lanes of one call, scalar masters from oneloopmaster cached per event. A second reduction in a relabelled frame reuses the numerators and the cached masters as the cheap stability check.

Counterterms. The model's UV and R_2 vertices, two-point ones included, are ordinary vertices of the tree recursion, and each contribution contains exactly one of them; external field factors are separate sources. Admission is the requested coupling orders, the exactly-once constraint and a public `particle_veto`: a coupling term whose loop particles include a vetoed particle is dropped, and a vetoed particle is removed from every tree vertex, loop line and propagator of the generation, so loops and counterterms stay consistent. Nothing is matched to the loop (user, 23 October; addendum J). On the slice the counterterms come from the WP2.2 recursion (one counterterm order in the tree recursion, the default route since a18b7599, whose $gg \to ggggg$ program leaves no rule to site closure, Table 12); it still admits counterterm sites by a witness table of divergent loop routings with a power-counting completeness check, as the site closure (the test oracle) does. The package `ct-simplify` removes the witness table and the matching from production and adds `particle_veto`, which replaces the internal top-free switch of Table 13 (then `particle_veto = [t]`); site closure and the witness machinery remain only as test oracles.

Precision, stability and assembly. Every kernel is generic over binary64, DoubleFloat, arbitrary-precision floats and exact rationals. A failed lane fails its point and never branches a kernel. The assembly takes $2\,\mathrm{Re}$ of $i/(16\pi^2)$ times the reduced family sum, plus local UV, R_2 and field terms, per sector and column, with optional rotation, reduction-direction and rescaling estimates and the finite-only mode. The public API selects `loop_engine = "v2"` from TOML, CLI or Python and stores plan v4 as a relocatable PACBIN member (zlib JSON of exact kernel term lists, no compiled code, digest checked at load).

Table 16: **Asymptotic classes per event** for n gluons, full colour (DESIGN §1.4): $F_n \approx (n-1)!\left(\frac{1}{\ln 2}\right)^n/2$ families, 2^n helicities. The required set ($n \le 7$) is far below the crossover of schedule B with the unitarity class ($n \approx 30$–50); schedule M is planned as the default from $n \ge 8$.

Stage or code	Class
Tree blocks	$\Theta(n \cdot n!)$ currents; $\Theta(n! \cdot 2^n)$ current-helicity rows
Dressing, schedule B	$\Theta((n-1)! \cdot 4^n)$ seed injection; trie dressing dominates for $n \le 8$
Dressing, schedule M	$\Theta(n! \cdot 2^n \cdot n^4)$
NinjaRS	$\Theta(F_n)$ reductions, $\Theta(F_n \cdot n^8)$ work, $\Theta(4^n)$ distinct masters after caching
MadLoop (per diagram, per helicity)	$\Theta(D_n \cdot 2^n \cdot n^4)$, D_n super-exponential ($gg \to gggg$: 26 975 loop diagrams)
Unitarity with exact helicity sums (NJet)	$\Theta((n-1)! \cdot 2^n \cdot n^{6\ldots9})$

Table 17: **Ownership of the mathematics.** pyAmpliCol reuses its dependencies for algebra, tensors, graphs and integrals and owns the recursions and the assembly.

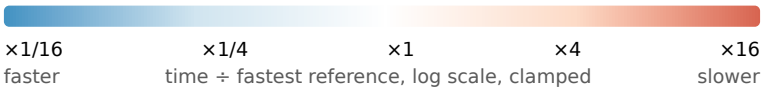
Component	Responsibility
ufo_model_loader	Lossless UFO loop-model import: Laurent-valued parameters and couplings, UV/R_2 categories, loop groups
Symbolica	Expressions, coefficient extraction in q, evaluators; exact rationals and arbitrary precision
Spenso, Idenso (Community)	Lorentz and colour tensors behind the kernel catalogue and the colour metrics
FeynKit (Community)	Test oracles: graph and family counts, overall factors, drawings (Figure 2)
NinjaRS, oneloopmaster	Integrand reduction with Laurent expansion; scalar masters in all carriers
pyAmpliCol tree engine	Blocks, Born, coupling orders and colour flow (Python lane; Rust recurrence)
pyAmpliCol loop v2	Families, skeletons, schedules, seed tables, counterterm program, plan v4, native executor, API

6 History: legacy per-diagram engine, 2026-10-18

History only. These timings measure the superseded per-diagram engine (per-graph coefficient programs, pairwise colour overlaps, whole-process symbolic projectors) through its Python coordinator with exact decimal input, on repeated three-point streams, against providers that kept their protocol settings (actual poles, stability checks and rescue). They are neither the gate protocol nor the v2 engine, include $2 \to 2$ processes, and are kept for reference without gate counts. Revisions fa599885 and two follow-ups; AMD EPYC 9754 128-Core Processor.

Table 18: **Legacy engine, µs per point** (2026-10-18), binary64, full colour, all three Laurent coefficients; median of five warm calls; multiplier: time ÷ the fastest timed reference of that row, coloured only where MadLoop is timed. Grey rows: generation (pyAmpliCol fresh process to plan; MadLoop export and compilation; Recola generation; OpenLoops setup of a prebuilt library). B: points per call.

Process	B	pyAmpliCol (legacy)		MadLoop		Recola		OpenLoops	
$u\bar{u} \to e^-e^+$ photon only · 1 loop graphs	1	584	×12.5	46.9	fastest	1 880	×40.1	no matched control	
	128	147	×3.2	46.5	fastest	1 780	×38.4	no matched control	
	gen.	2.43 s		198 s		0.135 s		—	
$u\bar{u} \to d\bar{d}$ QCD · 14 loop graphs	1	1 730	×6.9	251	fastest	2 650	×10.5	548	×2.2
	128	764	×3.0	254	fastest	2 670	×10.5	563	×2.2
	gen.	7.09 s		14.4 s		0.253 s		setup 0.0382 s	
$u\bar{u} \to t\bar{t}$ QCD · 14 loop graphs	1	2 010	×5.7	352	fastest	2 910	×8.3	655	×1.9
	128	745	×2.0	376	fastest	3 050	×8.1	686	×1.8
	gen.	6.39 s		17.1 s		1.12 s		setup 0.0103 s	
$u\bar{u} \to gg$ QCD · 45 loop graphs	1	6 830	×14.8	463	fastest	5 340	×11.5	1 700	×3.7
	128	3 100	×8.3	372	fastest	4 470	×12.0	1 700	×4.6
	gen.	21.2 s		29.1 s		0.183 s		setup 0.0149 s	
$gg \to gg$ QCD · 186 loop graphs	1	33 700	×13.0	2 590	fastest	36 100	×13.9	11 900	×4.6
	128	14 300	×6.9	2 070	fastest	33 200	×16.0	16 200	×7.8
	gen.	98.7 s		30.1 s		0.170 s		setup 0.0130 s	
$u\bar{u} \to ggg$ QCD · 556 loop graphs	1	194 000	×5.4	36 100	fastest	81 500	×2.3	36 900	×1.0
	128	125 000	×3.7	33 500	fastest	88 600	×2.6	43 400	×1.3
	gen.	863 s		23.9 s		0.724 s		setup 0.0109 s	
$e^-e^+ \to \mu^-\mu^+$ full EW, G_μ · 98 loop graphs	1	6 590	×2.1	validated, not timed		3 100	fastest	not timed	
	128	2 920	×1.0	validated, not timed		2 950	fastest	not timed	
	gen.	116 s		—		0.175 s		—	



The per-diagram design could not reach the $2 \to 5$ set: a seven-leg generation ran for more than nine hours without a result, and its counterterm enumeration alone needed up to 77 GiB. The v2 engine replaced its numerator, colour and counterterm core (Section 5) and keeps model import, the counterterm rules, Born weighting before reduction, the NinjaRS/oneloopmaster adapter, the exact carriers and the public interfaces.

7 Sources and reproducibility

- **Build:** `typst compile benchmarks.typ benchmarks.pdf` in `docs/performance_reports/loop/` (Typst 0.15).
- **Status, in-flight work, decisions:** `data/milestone.json`, from the `Integrate …` merges of dfa6bf7c since f9edf1db `workspace/reports/engine-v2/slice-status.md` (integration blocks, DECISIONS 2026-10-23); `HANDOFF_22_10_2026.md` §0–§3; MASTER_PLAN 2026-10-23 entries; REQUIREMENTS_ADDENDUM A, D, F, G, H, J; DESIGN §5.7 (G1), §6.1, §12.4.
- **Table 8:** provider columns HANDOFF §0.1 (addendum F); pyAmpliCol column DESIGN WP3.1 third increment "Generation per process" on `work/v2-generation-scale` (201efa8b). **Table 9:** DESIGN WP3.1 first to fourth increment, same branch (records `workspace/tmp/generation-scale/runs/`); the $gg \to gggg$ row from `cmp_gg_gggg.json`, `cmp_gg_gggg_p1_stab.json` and `cmp_gg_gggg_p1_dd.json` there. **Table 10:** DESIGN §1.3 Table 1b. **Conventions:** MadLoop standalone, Recola, OpenLoops 2.
- **Section 1 (Table 1 to Table 7):** `data/g1.json`, written by `python -m tools.loop_benchmark gate` (README and docstrings of `tools/loop_benchmark`); streams `tools/loop_benchmark/streams/`; records and logs `workspace/tmp/measure-g1/{runs,logs}/`; recalibration `tools/loop_v2_costmodel.py --g1 data/g1.json`; builds `loop-native-measure-g1-0f87abd2` (slice + timers) and `loop-native-measure-g1-genscale-39824f9b` (889af02c + timers), release with one codegen unit and thin LTO; schedule M at seven legs: DESIGN §5.6.
- **Table 11** and NJet prose: `workspace/reports/engine-v2/njet-benchmark.md` §2, §5, §6 (raw `workspace/reference/runs/njet/timing/`); NJet, arXiv:1209.0100 Table 5.
- **Table 12, Table 13, first virtual:** `data/measured.json`, written by `scripts/collect_milestone.py <workspace>` from the records `gen_gg_ggggg_final`, `gen_gg_ggggg_hash`, `poles2_gg_ggggg`, `eval2_gg_ggggg`, `eval2_gg_ggggg_rot` and `cmp_uux_ggggg` in `workspace/tmp/generation-scale/runs/` and `gg_ggggg` in `workspace/tmp/ct-recursion/s4/` (JSON and `/usr/bin/time` logs); thread settings from DESIGN WP3.1 and ct-recursion DESIGN §6.1; the 119/3 shift and NJet deviations from njet-benchmark.md §5.3 and the fixture `tests/fixtures/loop/required/gg_ggggg_njet.json`; top-free values from DESIGN WP3.1 fourth increment and `tests/unit/test_loop_v2_topfree.py` (`work/v2-generation-scale`). Memory is shown in GiB = KiB/2^{20}.
- **Table 14, Section 5:** `docs/development/loop_v2/DESIGN.md` §1–§3, §5.5, §5.6, §6, §10, §12, REQUIREMENTS_ADDENDUM J (counterterm rule) and the integration records above.
- **Table 18:** `data/measurements.json`, `data/provider-cold.json` (legacy snapshot of 2026-10-18, unchanged) and `data/legacy-labels.json`; `data/colour-measurements.json` stays for the DESIGN citations. Removed at this milestone, in the git history at `45875b87:docs/performance_reports/loop/data/`: `processes.json`, `provider-audit.json` and `generation.json` (the 9.4 h and 77 GiB of Section 6); see the README.